Self-Reflective Report

Full Name: NABID REAZUL ISLAM

Student ID: 202505010344

Programme: BCSSE

Subject: BIT1123 — Object Oriented Programming Fundamentals in Java

Institution: City University Malaysia, Faculty of Information Technology

GitHub: https://github.com/reazulislamnabid-oss/NABIDREAZULISLAMSTUDENTID202505010344

1. Introduction

This report reflects on my personal experience going through the BIT1123 Object-Oriented Programming (OOP) tutorials in Java this semester. Throughout these weekly hands-on sessions, I built a solid foundation in core Java programming and object-oriented principles. The following sections highlight what I picked up each week, the exact issues I ran into, and how I worked through them to build my skills.

2. Weekly Reflection

Week 1 — Introduction to Java

- **What I learned:** How Java actually runs under the hood (JDK, JRE, and JVM), basic syntax, primitive data types, and using Scanner alongside System.out.println() for basic inputs and outputs.
- **Challenges faced:** Moving to a strictly typed language was tricky at first, and I kept forgetting minor details in boilerplate code like public static void main(String[] args).
- **How I addressed them:** I wrote and ran small scripts repeatedly in the IDE, paying close attention to the compiler errors until writing the basic structure became second nature.

Week 2 — Classes and Objects

- **What I learned:** The core ideas behind OOP—defining classes, creating instances with new, using constructors, setting instance variables, and managing public vs. private access modifiers.
- **Challenges faced:** It was hard to picture how reference variables work in memory compared to standard primitive data types.

- **How I addressed them:** I drew quick heap and stack memory diagrams on paper to track how objects are stored, then built basic classes like Student and BankAccount to test things out.

Week 3–4 — Control Flow and Methods

- **What I learned:** Working with conditionals (if-else, switch), loops (for, while, do-while), writing custom methods, passing arguments, and overloading methods.

- **Challenges faced:** I occasionally got stuck in infinite loops or hit off-by-one errors when writing multi-layered loops, and wasn't always sure when to split code into separate methods.

- **How I addressed them:** I started tracing variables step-by-step on paper before running the code and made a point to break down long chunks of code into smaller, single-purpose methods.

Week 5 — Arrays

- **What I learned:** Declaring, initializing, and iterating through 1D and 2D arrays using standard for loops and for-each loops, plus basic array operations like finding min/max values.

- **Challenges faced:** Constantly running into ArrayIndexOutOfBoundsException errors, especially when trying to loop through 2D grids.

- **How I addressed them:** I stopped hardcoding array lengths and started relying strictly on array.length, while adding print statements inside nested loops to keep track of row and column indexes during debugging.

Week 6 — Strings

- **What I learned:** String immutability, how the String Pool saves memory, built-in operations like .substring(), .split(), and .indexOf(), and using StringBuilder for heavy text edits.

- **Challenges faced:** I kept making the rookie mistake of comparing strings with == instead of .equals(), which caused subtle bugs in logic checks.

- **How I addressed them:** I made it a strict habit to always use .equals() or .equalsIgnoreCase() for string checks, and switched over to StringBuilder whenever I needed to modify text inside loops.

Week 7–8 — Inheritance and Polymorphism

- **What I learned:** Using extends to share code across classes, overriding methods with @Override, dynamic method binding, and using super to call parent constructors.

- **Challenges faced:** Understanding how static types differ from dynamic types during runtime, and avoiding bad inheritance designs that didn't make logical sense.

- **How I addressed them:** I began checking object types with instanceof before doing any type casting to prevent crashes, and always double-checked that my child class had a genuine "is-a" relationship with the parent class.

Week 8–9 — Abstract Classes and Interfaces

- **What I learned:** Designing abstract classes with mixed concrete/abstract methods, using interfaces as contracts, implementing multiple interfaces, and using default methods.

- **Challenges faced:** Deciding when to use an abstract class versus when an interface made more sense for a project setup.

- **How I addressed them:** I set a straightforward rule for my designs: use abstract classes when objects closely share core state or logic, and use interfaces when totally unrelated classes simply need to share a specific action or behavior.

Week 10 — Exception Handling and File I/O

- **What I learned:** Catching runtime errors using try-catch-finally, handling checked vs. unchecked exceptions, and reading/writing plain text files with Scanner, File, and PrintWriter.

- **Challenges faced:** File streams getting left open whenever a program crashed unexpectedly, and accidentally catching broad Exception objects instead of real error types.

- **How I addressed them:** I started using try-with-resources so Java handles closing streams automatically, and began catching specific errors like FileNotFoundException to print clear, helpful messages.

3. Overall Challenges and How I Overcame Them

The biggest hurdle I faced throughout the module was switching from simple step-by-step programming to thinking in terms of objects and systems. Early on, I struggled to figure out how different classes should talk to each other without making the code messy. I solved this by sketching quick class diagrams on paper before jumping straight into typing code, breaking big problems down into small parts first. This taught me that spending extra time planning out code architecture upfront saves hours of frustrating debugging later on.

4. Key Takeaways

Through these tutorials, I have learned that:

1. **OOP is about organization:** Object-oriented programming is really about designing clean, reusable structures, not just stacking functions together.

2. **Modular code makes bug fixing easier:** When every class has one clear job, finding and fixing broken code takes a fraction of the time.

3. **Practice is everything:** Reading theoretical concepts only gets you so far; real confidence comes from fixing broken code and running projects directly in the IDE.

4. **Git makes project tracking simple:** Using version control tools like Git and GitHub helps keep work organized, prevents lost progress, and serves as a good portfolio for future work.

5. Conclusion

Overall, the BIT1123 tutorials gave me a solid, practical understanding of Java and object-oriented development. I feel much more comfortable building structured programs, designing clean classes, and managing source code with Git. What I've learned in this subject will definitely help me handle more advanced software development modules down the road.